DigMitra PRO — Technical & Architecture Report

VERSION 2.0 PRO
PRODUCTION READY

JCB Site Cleaning, Plot Leveling & Debris Extraction Marketplace Application

Complete engineering documentation covering programming languages, frontend/backend frameworks, geospatial mapping engines, API keys, cloud database credentials, consumer booking workflow, and codebase structure.

PRIMARY LANGUAGE	APP FRAMEWORK	CLOUD DATABASE	LOCAL / NETWORK PORT
TypeScript 7.0 (TSX/TS)	**React 19 + React Router 8**	**Supabase PostgreSQL**	**Port 5050 (Vite 8 SSR)**

1. PROJECT OVERVIEW & COMMERCIAL VISION

PRODUCT ARCHITECTURE

DigMitra PRO is a mobile-first, real-time heavy-machinery booking marketplace designed to bridge the gap between property owners/citizens and verified JCB/Excavator/Tipper Truck operators. Similar to how Uber/Rapido revolutionized passenger transport, DigMitra enables any citizen to pinpoint their plot via live hardware GPS, inspect nearby available JCBs, upload site photos, choose a payment mode, pay a nominal ₹500 token advance, and track their dispatched JCB live on a turn-by-turn map.

Target Market & Use Cases

- **Plot Clearing & Bush Removal** (30×40 ft, 40×60 ft, 60×40 ft residential/commercial sites)
- **Construction Debris Extraction** (JCB Backhoe + Tractor Tipper haulage combos)
- **Foundation Trenching & Site Leveling**
- **Primary Hub**: Kyatasandra, Tumakuru & Bengaluru, Karnataka

Multi-Role & Bilingual Capabilities

- **3 User Roles**: Client/Consumer, JCB Fleet Owner, and Marketplace Admin
- **Bilingual UI**: Instant 1-tap switch between **English** and **Kannada (ಕನ್ನಡ)**
- **Anti-Fraud Token Model**: ₹500 advance online lock + remaining balance via Cash on Spot or Online

2. PROGRAMMING LANGUAGES USED

LANGUAGES

LANGUAGE	VERSION / DIALECT	CODEBASE SHARE	WHERE & HOW IT IS USED IN DIGMITRA
TypeScript (`.tsx` , `.ts`)	TypeScript `^7.0.2`	**~92% Primary**	All React UI screens, interactive map components, state stores, booking workflows, translation dictionaries, Vite server configuration, and database migration scripts.
JavaScript (`.js` , `.mjs`)	ES2022+ / Node ESM	**~4% Runtime**	Node.js server runtime (`>=22.22.0`), Leaflet/MapLibre browser scripts, and client-side storage/theme initialization scripts.
SQL (PostgreSQL)	PostgreSQL 15 / 16	**~2% Data Layer**	Supabase cloud relational database tables (`vehicles` , `bookings` , `live_tracking`), Drizzle ORM schemas, and spatial coordinate indexing.
HTML5 & CSS3	Tailwind CSS v4.2.4	**~2% Styling**	`app/global.css` , responsive mobile shell viewport styling, custom map marker animations, glassmorphism cards, and print-ready PDF guides.

3. COMPLETE TECHNOLOGY STACK (FRONTEND, BACKEND, DATABASE & MAPS)

FULL STACK

LAYER / CATEGORY	TECHNOLOGY / PACKAGE	VERSION	PURPOSE IN PROJECT
Frontend UI Library	`react` & `react-dom`	`19.2.7`	Core component rendering, hooks, concurrent UI transitions, and interactive modals.
Routing & SSR Framework	`react-router` & `@react-router/dev`	`8.1.0`	Full-stack file-system routing (`_index.tsx` , `map.tsx` , `book.tsx`) and Server-Side Rendering.
Build & Dev Server	`vite` + `@agent-native/core`	`8.1.0` / `0.182.1`	Ultra-fast HMR bundler, custom API middleware, Permissions-Policy headers, and Nitro/H3 server.
Styling Engine	`tailwindcss` + `@tailwindcss/vite`	`4.2.4`	Utility-first responsive styling, dark/light badges, gradients, and mobile-first ergonomics.
Accessible UI Primitives	`@radix-ui/react-*` + `vaul`	`Latest`	Dialogs, dropdowns, sheets, tooltips, accordions, switches, and mobile bottom drawers.
Icon System	`@tabler/icons-react`	`3.41.1`	Crisp vector icons for machinery, GPS pins, camera, phone calls, shields, and navigation.

LAYER / CATEGORY	TECHNOLOGY / PACKAGE	VERSION	PURPOSE IN PROJECT
Geospatial Map Engine	`maplibre-gl` + `react-map-gl` + `Leaflet`	`6.11.2` / `8.1.3`	WebGL vector tiles, satellite imagery switching, draggable site pins, and live JCB radar markers.
State Management	`zustand` + `@tanstack/react-query`	`5.0.15` / `5.99.2`	Reactive client state, real-time in-app push notification store, and async server cache.
Forms & Validation	`react-hook-form` + `zod`	`7.71.2` / `4.4.3`	Form state management and strict TypeScript runtime validation for phone numbers, OTPs, and bookings.
Database & ORM	`@supabase/supabase-js` + `drizzle-orm` + `postgres`	`2.117.2` / `0.45.2`	PostgreSQL cloud connection pooling, real-time subscriptions, and type-safe SQL queries.
AI & Agentic SDKs	`ai` + `@ai-sdk/google` / `openai` / `anthropic`	`6.0.258`	Integrated AI assistant capabilities and multi-model LLM readiness.
Package Manager & Runtime	`pnpm` + `Node.js` + `tsx`	`10.14.0` / `>=22.22`	Deterministic dependency management and TypeScript script execution.

4. ALL API KEYS, DATABASE CREDENTIALS & EXTERNAL SERVICES USED

API KEYS & SECRETS

Below is the complete inventory of all environment variables, API keys, database connection strings (stored in `.env`), and external geospatial APIs active in the project:

```
# 1. SUPABASE CLOUD POSTGRESQL (TRANSACTION POOLER - PORT 6543)
DATABASE_URL=postgresql://postgres.avavshltiiafnxdzrvza:Abhi%407892667467@aws-0-ap-south-1.pooler.supabase.com:6543/postgres

# 2. SUPABASE CLOUD POSTGRESQL (SESSION / DIRECT POOLER - PORT 5432)
DATABASE_URL_UNPOOLED=postgresql://postgres.avavshltiiafnxdzrvza:Abhi%407892667467@aws-0-ap-south-1.pooler.supabase.com:5432/postgres

# 3. SUPABASE PROJECT API URL (AP-SOUTH-1 MUMBAI REGION)
VITE_SUPABASE_URL=https://avavshltiiafnxdzrvza.supabase.co

# 4. STADIA MAPS VECTOR TILE & STYLE API KEY
VITE_STADIA_KEY=abd1cf7c-c8ca-4b47-a786-de5a7d41aaa6
```

SERVICE / API NAME	KEY / IDENTIFIER / ENDPOINT	AUTH TYPE	EXACT ROLE IN APPLICATION
Stadia Maps API	Key: `abd1cf7c-c8ca-4b47-a786-de5a7d41aaa6` Style: `alidade_smooth.json`	API Key	Renders high-contrast street vector maps and road networks in `realtime-map.tsx` and `live-tracking-screen.tsx`.
Supabase PostgreSQL	Project Ref: `avavshltiiafnxdzrvza` Host: `aws-0-ap-south-1.pooler.supabase.com` Password: `Abhi@7892667467`	PostgreSQL URI	Cloud relational database storing JCB fleet vehicles, consumer bookings, advance payments, and live GPS coordinates.

SERVICE / API NAME	KEY / IDENTIFIER / ENDPOINT	AUTH TYPE	EXACT ROLE IN APPLICATION
Supabase JS Client	URL: `https://avavshltiiafnxdzrvza.supabase.co` Env: `VITE_SUPABASE_ANON_KEY`	**JWT Anon Key**	Client-side Supabase SDK configured in `app/lib/supabase.ts` for direct browser queries & realtime channels.
Esri ArcGIS World Imagery	`https://server.arcgisonline.com/ArcGIS/rest/services/World_Imagery/MapServer/tile/{z}/{y}/{x}`	**Public Tile API**	Provides high-resolution aerial satellite view when the user clicks the **"Satellite"** toggle on the map.
OpenStreetMap Nominatim	`https://nominatim.openstreetmap.org/reverse?format=json&lat={lat}&lon={lng}`	**Public REST API**	Converts raw hardware GPS coordinates (Latitude/Longitude) into human-readable street & village names.
Browser Hardware GNSS / GPS	`navigator.geolocation.getCurrentPosition` & `watchPosition`	**Permissions-Policy**	Fetches real-time hardware satellite/Wi-Fi location from the user's phone or laptop (`geolocation=*` enabled).
Browser MediaDevices Camera API	`navigator.mediaDevices.getUserMedia({ video: { facingMode: "environment" } })`	**Permissions-Policy**	Powers the live plot inspection camera in `live-camera-modal.tsx` (`camera=*` enabled).

5. END-TO-END CONSUMER BOOKING ARCHITECTURE (7-STAGE FUNNEL) USER JOURNEY

1. **Login & Instant OTP Verification (`app/components/auth/login-screen.tsx`)**
   Displays the clean welcome header ("*Hello Client...!! Ready to book JCB...*" with custom JCB icon), collects **Name** and **10-digit Mobile Number (+91)**, generates a 4-digit OTP with an interactive SMS Banner & 1-click Auto-Fill, and includes a plain-text "*Log-in as JCB Owner*" mode switcher at the bottom.

2. **Live Hardware GPS Map & JCB Discovery (`app/components/map/realtime-map.tsx`)**
   Automatically acquires the user's real-time hardware location using a 3-tier GPS engine (Satellite GNSS → Network Wi-Fi Fix → Calibrated Kyatasandra Hub). Displays nearby JCBs with live ETA badges, supports Vector/Satellite switching, and presents the bottom confirmation card with the prominent yellow **"BOOK JCB →"** button.

3. **Site Booking Details & Live Plot Camera (`app/components/booking/site-booking-details.tsx`)**
   Allows the consumer to select plot dimensions (`30 × 40 ft` , `40 × 60 ft` , `60 × 40 ft` , or custom), adjust estimated machine hours, toggle **Tractor Tipper** debris removal trips, and capture/upload surrounding plot photos via the live camera modal.

4. **Payment Mode Selection (`app/components/booking/payment-mode-screen.tsx`)**
   Lets the consumer choose how they want to settle the main work bill: **Cash on Spot** (pay operator directly at the site after work completion) or **Online Payment** (UPI / NetBanking / Cards).

5 **JCB Machine Catalog & Operator Selection (`app/components/booking/jcb-catalog-screen.tsx`)**
Presents available machinery near the site: **JCB 3DX Super Backhoe (₹950/hr)**, **JCB 3DX EcoXpert (₹1,050/hr)**, **JCB 2DX Mini Backhoe (₹800/hr)**, and **Heavy Tracked Excavator (₹1,450/hr)** with operator ratings, registration numbers, and ETAs.

6 **₹500 Token Advance Payment Gateway (`app/components/booking/advance-payment-screen.tsx`)**
Collects a refundable/adjustable **₹500 Advance Booking Fee** via PhonePe, Google Pay, Paytm, or UPI ID to confirm genuine intent and lock the JCB operator. Automatically deducts ₹500 from the total estimated bill.

7 **Booking Confirmation & Live Turn-by-Turn Tracking (`booking-success-screen.tsx` & `live-tracking-screen.tsx`)**
Displays the high-visibility green/yellow confirmation modal with Booking ID (`#JCB-XXXX`), triggers an in-app push notification with sound, and launches the live route tracker showing the JCB moving in real time toward the consumer's plot.

6. PROJECT DIRECTORY & KEY SOURCE FILES

CODEBASE STRUCTURE

FILE PATH	MODULE RESPONSIBILITY
`vite.config.ts`	Port 5050 server config, `Permissions-Policy` geolocation header injection, and custom API endpoints (`/api/location` , `/api/report-location` , `/api/latest-device-gps`).
`.env`	Stores `DATABASE_URL` , `DATABASE_URL_UNPOOLED` , `VITE_SUPABASE_URL` , and `VITE_STADIA_KEY` .
`app/routes/_index.tsx`	Main application orchestrator managing authentication state, language switching, and the 7-step consumer booking funnel.
`app/routes/map.tsx` & `app/components/map/MapView.tsx`	Standalone high-precision MapLibre GL hardware GPS watcher route (`/map`).
`app/components/auth/login-screen.tsx`	Consumer & JCB Owner login screen with custom JCB emoji (`/jcb-emoji.png`) and 4-digit OTP verification.
`app/components/map/realtime-map.tsx`	Interactive Consumer Map with 3-tier hardware GPS, Stadia/Satellite tiles, JCB markers, and Book JCB bottom card.
`app/components/booking/*`	Contains `site-booking-details.tsx` , `live-camera-modal.tsx` , `payment-mode-screen.tsx` , `jcb-catalog-screen.tsx` , `advance-payment-screen.tsx` , and `booking-success-screen.tsx` .
`app/components/mobile/*`	Contains `mobile-shell.tsx` , `owner-dashboard.tsx` (for JCB owners), `admin-dashboard.tsx` , `site-calculator.tsx` , and `support-sheet.tsx` .
`app/lib/translations.ts`	Complete bilingual English (`en`) and Kannada (`kn`) dictionary across all screens.